# Removing Electrical Noise Coupling via Deep Learning Techniques

Kyle Hildebrand  Jah Multani  Jevin Barnett
Michigan Technological University
CS5841 Final Project

# Problem Statement

- Electrical signals in Electronic Control Units can have noise that conflict with signal integrity and control
- Electrical signals can have fast rising and falling edges that can corrupt lower energy signals
- Electrical noise is defined as unwanted data on a specific signal
- Deep Learning techniques were studied and compared to classical methods like a digital infinite impulse response
- To use on an embedded system a forward pass was calculated on an STM32 and the parameter of the model should be small to account for computation time on microcontroller

# Background

- A few approaches have already been taken for removing noise from signals using neural networks
- Donoho introduced wavelet soft-thresholding as a method for reconstructing a clean signal by transforming a noisy observation into the wavelet domain and shrinking coefficients associated with noise. [1]
- Alkhafaji et al. developed a lightweight one-dimensional convolutional neural network for the joint classification and denoising of communication signals in low-SNR edge environments [2]
- Sun et al. proposed an end-to-end one-dimensional residual convolutional neural network for removing artifacts from EEG signals [3]

# Motivation

- The motivation for the development of a neural network that removes noise from a real time signal is to explore options for future signal processing
- Various neural network architectures can be used for comparison, SNR (signal to noise ratio) is used to determine how well each model performs
- If it is proven that a neural network can effectively and efficiently remove the noise from a signal, then it can be employed in a number of settings

# Method Overview

- Three deep learning techniques were trained on various sets of data
- Multilayer Perceptron, 1D Convolutional Neural Network and Denoising Autoencoder were trained and evaluated
- Data was collected from a real life ECU exhibiting noise at consistent sample rate
- Noisy data was trained against a baseline through the various models
- An Infinite Impulse Response filter was used as a comparison
- MLP forward pass ran on the Stm32 microcontroller

# Pipeline / Workflow

- Data is collected by recording noisy ECU data from an automotive setting
- The data is processed to clean up discontinuous or missing data
- Google drive was used with Jupyter notebooks to prepare the data for training
  - Train, test, validation splits were created
  - Sliding windows to provide input data to the model was created
- Classical Baselines:
  - Rolling average with 15 samples
  - IIR filter
- Networks built:
  - MLP
  - 1D CNN
  - LSTM
  - Autoencoder

# Model Details

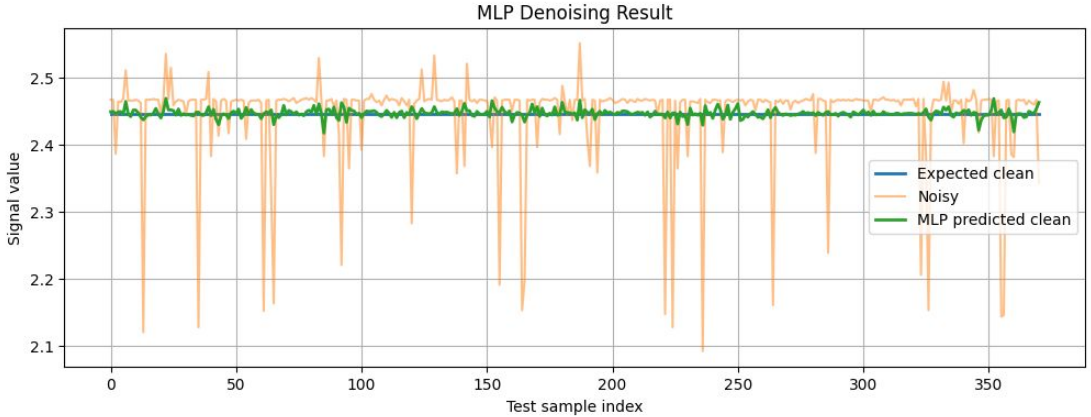
| Model | Architecture | Parameters | Performance | | |
|---|---|---|---|---|---|
| | | | SNR | MSE | MAE |
| MLP | Dense layers with relu | 8,385 | 51.48 | 0.00004252 | 0.00437203 |
| 1D CNN | Convolution layers with dense output | 5,921 | 31.11 | 0.00463302 | 0.03185454 |
| LSTM | LSTM layers with relu activation | 29,345 | 74.29 | 0.00000022 | 0.00026900 |
| Autoencoder | Convolutional encoder and decoder | 67,105 | 52.56 | 0.00003319 | 0.00381722 |

# Experimental Setup

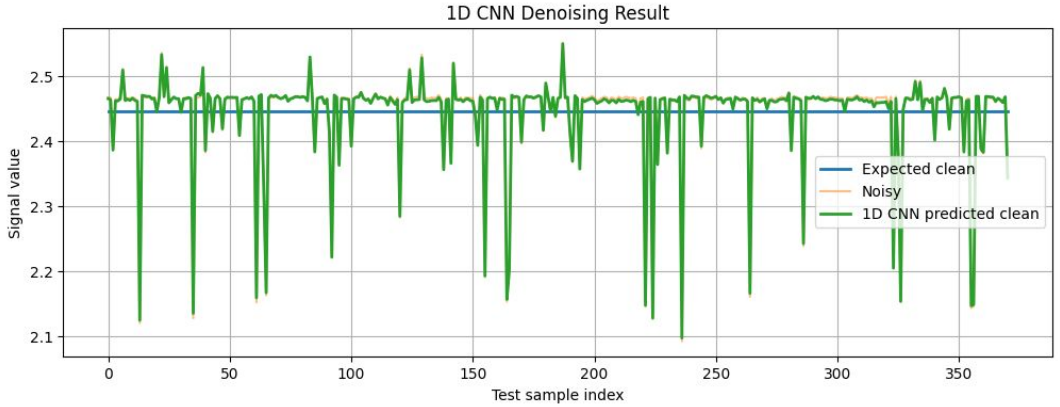
- Data collection
- Each model was trained on the input data set
- Based on the results from the training, the MLP and the autoencoder were selected to be run on the STM32 development board
- Models were quantized and converted to tflite models, then loaded into the STMCubeMX tool
- Running a forward pass on a set of test inputs results in a measurable runtime that varies based on the number of parameters used

# Results Pt. 1 - MLP and 1-D CNN

- MLP predicted output showed an SNR improvement of about 20 dB
- Training is able to get consistent results by minimizing the loss (MSE)
- Issues with this model could include overgeneralization, where the model only learns to output a single value
- Forward pass on a quantized model ran at 1.7ms on the STM32

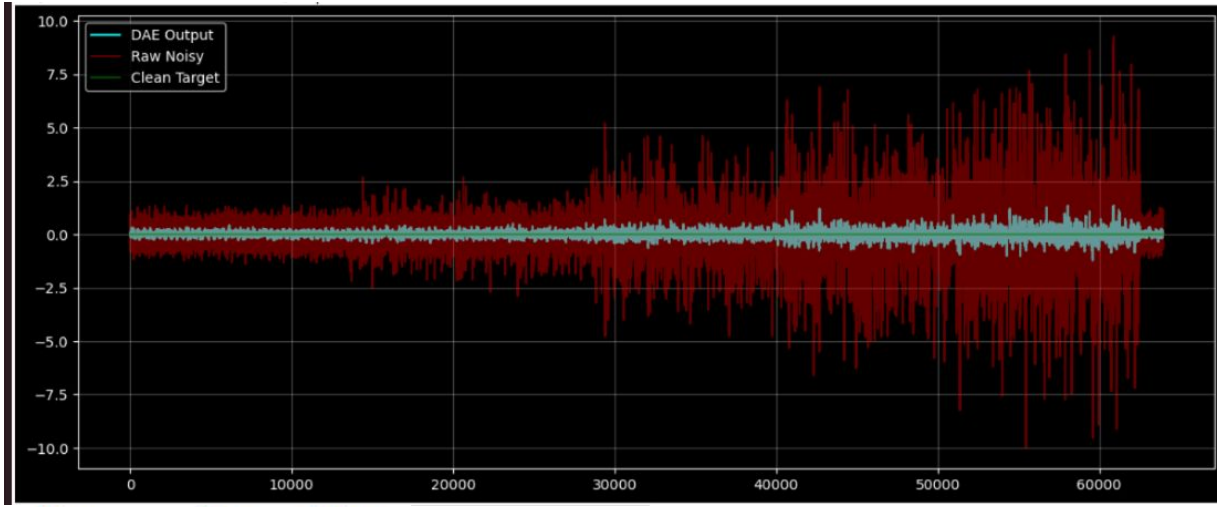



- 1-D CNN predicted output was overfit to the noisy data
- Training error was significantly reduced, but the model was not able to remove the noise, with an SNR of 31 dB, equal to the baseline
- This model was not selected to run on the STM as it did not perform well



# Results Pt. 2 Denoising Autoencoder

- Output had ~10DB improvement over input data
- DAE improved the min/max implying its better at removing extremes
- The Variance and standard deviation were comparable
- DAE had a lower mean than the IIR




| | Model | Mean | Standard Dev | Var | Min | Max |
|---|---|---|---|---|---|---|
| 0 | Baseline | 0.009477 | 0.644937 | 0.415944 | -10.005383 | 9.271493 |
| 1 | DAE | 0.009450 | 0.183386 | 0.033630 | -1.226472 | 1.345005 |
| 2 | IIR | 0.009469 | 0.182215 | 0.033202 | -1.579171 | 1.746068 |

# Conclusion

- Neural Networks are an effective way to remove a noise from a known signal
  - The tricky part is getting it to be reference aware, which was not observed with many of the models used
- DAE had a better mean (closer to 0) and a lower min/max than the IIR
- The IIR and DAE had a similar standard deviation and variance
- DAE loss converged and showed a 10db improvement
- STM32 forward pass for MLP was 1.7ms for 1 forward pass of the MLP and about .85ms for 1 forward pass of the DAE

# Future Work and Q+A

- Data samples used contained 2500 and two sets of 65000 samples. Need to expand to a much larger dataset